

NETFRAME

Operations Documentation **Bare-Metal Build Guide**

Phased rebuild-from-nothing procedure, in dependency order,
with an explicit validation gate at the end of every phase

Document ID	NF-RC-010
Owner	Infrastructure Engineering
Classification	Public (redacted)
Last updated	2026-07-13
Related	NF-RC-002, NF-RC-001, NF-AR-001

A redacted public sample from the NetFRAME operations library.
Demonstrates operational-documentation discipline; not an operational document.

github.com/machismo0311/netframe-ops-docs-public

2026 • Redacted public edition • full internal edition kept private

Contents

1. How to Use This Guide

2. Phase 0: Out-of-Band Secrets Bootstrap (do this first)

3. Phase 1: Bare Metal and Proxmox VE

4. Phase 2: Cluster Formation and Quorum

5. Phase 3: The OPNsense Edge (network first)

6. Phase 4: DNS (Pi-hole primary and secondary)

7. Phase 5: Storage, PBS, and NFS (STORAGE)

8. Phase 6: Core LXC Services

9. Phase 7: RKE2 Kubernetes

10. Phase 8: GPU Nodes (GPU-A and GPU-B)

11. Final Cutover Validation

12. Troubleshooting

2

2

3

3

4

4

5

5

5

6

6

7

2.1 What the bundle contains

2.2 Procedure

2.3 Validation gate

3.1 Procedure

3.2 Validation gate

4.1 Procedure

4.2 Validation gate

5.1 Procedure

5.2 Validation gate

6.1 Procedure

6.2 Validation gate

7.1 Procedure

7.2 Validation gate

8.1 Procedure

8.2 Validation gate

9.1 Procedure

9.2 Validation gate

10.1 Procedure

10.2 Procedure note: apply hardening at rebuild

10.3 Validation gate

2

3

3

3

4

4

5

5

5

6

6

6

6

6

6

Field	Value
Document ID	NF-RC-010
Owner	Infrastructure Engineering
Classification	PUBLIC (redacted)
Last updated	2026-07-13
Related documents	NF-RC-002, NF-RC-001, NF-AR-001, NF-RF-010, NF-RF-011, NF-OP-001

This is the Markdown edition of the guide, provided so it renders natively on GitHub. The **PDF edition** is the type-set original; both carry the same content. This is a **redacted public sample**: addresses and hostnames are generalized to roles (GPU-A, STORAGE, NODE-N), and container/VM IDs, service ports, exact software and kernel versions,

internal repository names, and named current-state gaps are omitted. It demonstrates operational-documentation discipline; it is not an operational document. The full internal edition is kept private.

Purpose. The authoritative, phased sequence to reconstruct the entire NetFRAME cluster environment from bare metal, in dependency order, with an explicit validation gate at the end of every phase. This is the single document a senior engineer follows to rebuild the platform from nothing.

Scope. Covers the build order and the dependency and validation logic that ties the phases together: bare-metal Proxmox, cluster and quorum, the OPNsense edge, DNS, storage and backups, core LXC services, RKE2 Kubernetes, and the GPU compute nodes. It deliberately does not replace the detailed per-component runbooks; it sequences them and states what must be true before each one runs.

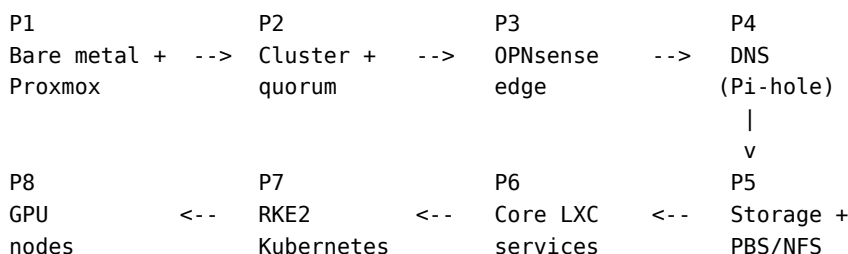
Assumptions. A full or partial rebuild is authorized. Hardware is physically present, cabled per the rack and network references, and powered. The out-of-band secrets bundle (see Phase 0) is available offline. The pinned Proxmox VE release install media is prepared.

Prerequisites. Console or KVM access to each node, the OOB secrets bundle and its age key, the most recent PBS datastore (or a STORAGE rebuild path), the infrastructure Git repositories (configuration-as-code, monitoring, and topology references), and physical or serial console to OPNsense. Read NF-RC-002 (Disaster Recovery) first for RTO/RPO intent and the decision of full-rebuild versus targeted restore.

1. How to Use This Guide

This guide is ordered by hard dependency, not by convenience. Each phase produces a platform layer that the next phase requires. Do not start a phase until the prior phase's validation gate passes; a gate that does not pass is a stop-and-diagnose point, not a proceed-anyway point. The environment's operating discipline is one change at a time, verify, then proceed; a rebuild follows the same discipline at the granularity of a phase.

The dependency spine is:



Phase 0 (secrets bootstrap) runs before Phase 1 and is out-of-band by design. Phases 5 through 8 read as a chain here for clarity, but storage (P5) is a hard prerequisite for both core services (P6) and Kubernetes persistent storage (P7).

2. Phase 0: Out-of-Band Secrets Bootstrap (do this first)

Caution. This phase exists to break a circular dependency. Every credential in the environment lives in the secret store, which runs as a container inside the cluster being rebuilt. You cannot log in to restore the secret store using a secret that is stored in the secret store. The bundle described here is the only way in.

2.1 What the bundle contains

An offline, encrypted bootstrap bundle, held on removable media (and a second copy off-site), independent of STORAGE and the cluster:

- The age private key used to decrypt the OPNsense config backup repo and other age-encrypted material.
- The secret store admin token and database export (or its restic/PBS restore path plus the passphrase to open it).
- The admin-workstation restic repository password (the off-box backup will not open without it).
- Root or console credentials for the initial Proxmox nodes and the OPNsense serial console.

- PBS datastore encryption key(s) if the datastore is encrypted, and the PBS fingerprint.
- The tailnet control server's pre-auth/registration key material, or the note that keys are regenerated on rebuild.

2.2 Procedure

1. Retrieve the removable media (or off-site copy). Verify integrity before trusting it.
2. Decrypt with age on a trusted admin workstation, then extract to a tmpfs you will wipe.
3. Confirm you can read: the OPNsense config, the secret store restore path, the restic password, and PBS keys. If any is missing, stop; a rebuild without these has no clean path back to the secret store.

2.3 Validation gate

You hold, in hand and verified, the age key, the secret store bootstrap secret, the restic password, and the PBS/OPNsense credentials before any node is reimaged. Do not proceed to Phase 1 until this is true.

Note. Maintaining this bundle is a standing operational task, not a rebuild-time task. Refresh it whenever the age key, the secret store token, restic password, or PBS key changes. See NF-RC-001 (Backup and Restore) and NF-RC-002 (DR).

3. Phase 1: Bare Metal and Proxmox VE

Depends on: Phase 0. **Produces:** each node running Proxmox VE, reachable on the management VLAN.

3.1 Procedure

1. **Firmware and BIOS.** Set BIOS to the recorded baseline: virtualization (VT-x/VT-d) enabled, correct boot order, power-on-after-AC-loss for the servers. On the servers (GPU-A, GPU-B, STORAGE), confirm out-of-band management (iDRAC/IPMI) is on the OOB VLAN and reachable. Bring firmware to the documented baseline where it is behind (this is also a named assessment gap).
2. **Storage controllers.** STORAGE presents a RAID1 boot volume; the datastore and bulk disks are JBOD/HBA for ZFS. The GPU nodes present their disks in HBA/JBOD so ZFS owns them directly. Do not create controller RAID over ZFS data disks.
3. **Install Proxmox VE** on each node in turn (the pinned release). Match the hostname exactly (NODE-2 to NODE-5, GPU-A, GPU-B, STORAGE); naming is load-bearing (see NF-RF-030). Install to the boot device only.
4. **Management networking.** Assign each node its management-VLAN address per NF-RF-010 (IPAM). One node carries a vlan-aware bridge trunk; the flat nodes sit on the management VLAN only. On the three servers, also configure the 10G storage-VLAN interface (dual-homed: management and corosync on the management VLAN, NFS/PBS/egress on the storage VLAN).
5. **Base config.** Apply the SSH hardening drop-in and baseline per NF-RF-031 (Configuration Standards). Set the baseline MTU (jumbo frames are not yet in the baseline). Do not yet pin the GPU-node kernel (that is Phase 8).

3.2 Validation gate

Every node boots Proxmox, is reachable by SSH on its management address, has correct hostname and time sync, and the servers can reach the storage VLAN. The version check reports the pinned release. No node yet references any other node.

4. Phase 2: Cluster Formation and Quorum

Depends on: Phase 1 (all nodes up, management network working). **Produces:** a quorate 7-node cluster with corosync knot.

4.1 Procedure

1. On the first node, `pvecm create` the cluster. Corosync uses the management network (the mgmt/corosync ring); keep it off the storage VLAN.
2. Join the remaining nodes one at a time with `pvecm add <first-node-ip>`, verifying quorum after each join before adding the next. Target the recorded config version and a 7/7 membership with quorum 4.
3. Confirm corosync transport is knot (secure). Leave HA fencing in standby (watchdog present, no HA-managed re-

sources) to match the current design; arming HA is a later, deliberate change, not part of the base rebuild.

4.2 Validation gate

`pvecm status` shows 7/7 members, quorate, quorum 4, `knet secure`, config version at the expected value. `corosync-cfgtool -s` shows all rings healthy. Do not proceed until the cluster is quorate; everything downstream assumes a stable control plane.

Note. NODE-1 (standalone Mac mini, Pi-hole primary) is not a cluster member. It is standalone and is rebuilt independently in Phase 4. Do not attempt to join it.

5. Phase 3: The OPNsense Edge (network first)

Depends on: Phase 2 (need a cluster node to host the OPNsense VM). **Produces:** routing, dual-WAN, seven VLANs, DHCP, and the single recursion point. Nothing beyond the cluster works without this.

Caution. OPNsense is the environment's largest single point of failure: one VM on one node carrying routing, inter-VLAN, DHCP, and DNS recursion. Build it deliberately and validate hard. See NF-RC-002 for the fast cold-restore path.

5.1 Procedure

1. Create the OPNsense VM on a cluster node and install from ISO, or restore from the PBS OPNsense backup if it is intact (restore is faster and lower-risk than rebuild-from-ISO).
2. Map interfaces exactly: WAN1 (primary ISP), the LAN trunk (10Gbase-T, 802.1Q, carries all seven VLANs), and WAN2 (cellular failover). Getting the interface mapping wrong is the most common rebuild error.
3. Import the OPNsense configuration from the age-encrypted config backup repo (decrypted with the Phase 0 age key). This restores the seven VLAN interfaces and per-VLAN gateways, the pf ruleset (default-deny, per-interface anti-spoof, dual-WAN outbound NAT), the DHCP scopes (every VLAN hands out both resolvers), and the single recursive resolver.
4. Verify both WAN gateways come up and outbound NAT is present on both. Failover monitors a public anchor.

5.2 Validation gate

From a host on the management VLAN: default gateway reachable, internet reachable via WAN1, inter-VLAN policy enforcing default-deny with explicit allows, DHCP leasing on each scope, and the recursive resolver answering. pf state table healthy. See NF-RF-011 (VLAN Reference) for the expected per-VLAN map.

Note. Firewall edits use the OPNsense automation-filter API (rules there evaluate before manual interface rules). If the GUI/API is unreachable during rebuild, the serial console is the fallback (verified Tier-A restore path).

6. Phase 4: DNS (Pi-hole primary and secondary)

Depends on: Phase 3 (edge recursion must answer). **Produces:** filtered, HA DNS on two resolvers.

6.1 Procedure

1. Rebuild the primary Pi-hole on NODE-1 (standalone Mac mini). This is off-cluster; it is rebuilt by its own procedure, not via Proxmox.
2. Rebuild the secondary Pi-hole as a container on a cluster node (restore from PBS if available).
3. Point both Pi-holes' upstream to the single recursive resolver at the edge. Restore the sync mirror so the secondary tracks the primary, and confirm DHCP failover coverage across VLANs.

6.2 Validation gate

Both resolvers answer external and internal names, both forward to the edge resolver, and the sync mirror shows the secondary in sync. Clients on every VLAN receive both resolvers via DHCP.

Caution. Both Pi-holes forward to one recursive resolver; a reload there deafens both at once for uncached lookups (a known single point of failure). During rebuild, expect brief resolution gaps whenever OPNsense reloads. The planned fix (a per-Pi-hole local resolver) is a post-rebuild improvement, not part of the base order.

7. Phase 5: Storage, PBS, and NFS (STORAGE)

Depends on: Phase 2 (cluster) and Phase 3/4 (network + DNS for STORAGE's services). **Produces:** ZFS pools, Proxmox Backup Server, and the NFS exports that core services and Kubernetes require.

7.1 Procedure

1. Import or rebuild STORAGE's ZFS pools: the internal datastore pool and the external-shelf bulk pool (dual-path multipath). Set `ashift=12`. Confirm multipath on the shelf before trusting the bulk pool. See STORAGE-commissioning-runbook.
2. Stand up Proxmox Backup Server on STORAGE. Restore or recreate the datastore (apply the Phase 0 PBS encryption key/fingerprint). This must exist before you can restore any guest from PBS in later phases.
3. Create the NFS exports on the storage VLAN, including the GPU-A export and the RKE2 NFS StorageClass export. Restore the admin-workstation restic repo target on the bulk pool.
4. Re-establish the backup schedule (staggered nightly windows, with the Wazuh backup over 10G; retention 7d/4w) and the snapshot replication. See PBS-10G-Backup-Path.

7.2 Validation gate

Both pools ONLINE (a scrub started or clean), PBS reachable with its datastore mounted, NFS exports mountable from a cluster node over the storage VLAN, and a test restore of one small guest from PBS succeeds. An untested restore path is not a validated one.

Note. STORAGE is the second keystone SPOF: all backups plus all Kubernetes NFS and the registry PV depend on it. If STORAGE itself is being rebuilt, this phase is on the critical path for everything stateful downstream.

8. Phase 6: Core LXC Services

Depends on: Phase 2 (cluster), Phase 4 (DNS), Phase 5 (storage/PBS for restore). **Produces:** the proxy, secret store, observability, tailnet, and portal.

8.1 Procedure

Rebuild or PBS-restore the core LXC stack in this order, because later services depend on earlier ones:

1. **The secret store** first. Restore its database from the Phase 0 bundle path (PBS/restic + passphrase). Once the secret store is up and unlocked, the circular dependency is broken and every other secret is available online. This is the moment the rebuild stops depending on the offline bundle.
2. **The reverse proxy** (nginx-proxy-manager) for reverse-proxy/TLS front ends.
3. **Observability** (Grafana + Prometheus + Loki). Restore config-as-code from the monitoring repo; bring the node exporters and log shippers on all nodes back online.
4. **The tailnet control server** (re-register nodes per its migration procedure).
5. **The portal services** (Homepage and the chat UI).

8.2 Validation gate

The secret store unlocks and serves credentials; the proxy serves its hosts with valid TLS; Grafana shows live metrics from all nodes and Loki is ingesting; the tailnet re-admits nodes; alerting fires on a test. Secrets are now sourced online, not from the offline bundle.

9. Phase 7: RKE2 Kubernetes

Depends on: Phase 5 (STORAGE NFS + local-ZFS PV for the registry) and Phase 4/6 (DNS + proxy/TLS). **Produces:** the RKE2 cluster, CNI, load balancing, and the private registry.

9.1 Procedure

1. Rebuild the three control-plane VMs with the kube-vip control-plane VIP. Install RKE2 (etcd co-located). See RKE2-Phase1-HA-ControlPlane.
2. Join STORAGE as a tainted bare-metal storage worker (note the kernel version drift versus the CP VMs; that is expected).
3. Deploy the CNI (with kube-proxy) and the L2 load balancer (a small address pool).
4. Restore the NFS StorageClass (default) and the static Retain PV for the registry on STORAGE local-ZFS. Bring up the private registry with internal-CA TLS and its cert-renew CronJob; establish node/containerd trust for the CA on all nodes.
5. Restore workloads.

9.2 Validation gate

All nodes Ready, no unhealthy pods, the control-plane VIP answers, the load balancer assigns its pool, the registry serves over TLS with a valid (non-expired) cert, and a test image pull/push succeeds. Dynamic PVC provisioning against the NFS StorageClass works.

Caution. The registry runs on the cluster it serves (a greenfield chicken-and-egg). On a cold rebuild you may need to pull base images from an external source until the local registry is serving. Adding registry authentication and a second replica is tracked as post-rebuild hardening.

10. Phase 8: GPU Nodes (GPU-A and GPU-B)

Depends on: Phases 2 to 6 (cluster, network, DNS, storage, monitoring). **Produces:** GPU-backed SLURM and LLM inference.

10.1 Procedure

1. Pin the GPU-node kernel to the driver-ABI baseline on both GPU nodes and hold it so a dist-upgrade cannot move it (the pin-without-hold gap is a named finding; hold it explicitly during rebuild). See NF-RF-031 and ADR-0010.
2. Install the pinned NVIDIA driver and CUDA. Enable persistence mode and confirm ECC on. GPUs bind to the host nvidia driver (bare-metal, no VFIO).
3. GPU-A: restore SLURM (controller, workers, accounting DB, and munge), the ZFS-backed containerd store, the local model-serving and vector stores, and Apptainer. Import the workspace pool (note: a legacy `ashift` value historically; a rebuild is the opportunity to correct to `ashift=12`). Mount the STORAGE NFS export.
4. GPU-B: restore bare-metal model serving, the LLM router, the monitor timer, GPU fan control, and the on-call bot. Import its ZFS pools.
5. Bring the Wazuh SIEM VM back on GPU-A and re-point agents fleet-wide.

10.2 Procedure note: apply hardening at rebuild

This is the natural point to close known findings rather than faithfully reproduce them: hold the kernel (done above), enforce key-only SSH on the GPU nodes, and bind or firewall any services that should not be broadly reachable. See NF-RF-031.

10.3 Validation gate

`nvidia-smi` shows the expected GPUs with persistence on and ECC on; the pinned kernel is running and held; SLURM accepts and runs a test job on GPU-A; GPU-B serves an inference request; the Wazuh manager is up with agents reporting.

11. Final Cutover Validation

After Phase 8, run the whole-environment gate before declaring the rebuild complete:

- Cluster 7/7 quorate; zero failed systemd units fleet-wide.
- Both WAN gateways online; inter-VLAN policy enforcing; DHCP and DNS (both resolvers + edge recursion) all answering.
- All ZFS pools ONLINE; PBS reachable; a test restore verified.
- RKE2 all-Ready; registry serving; dynamic PVCs work.

- GPUs healthy with pinned+held kernel; SLURM and inference working.
- Grafana green with all exporters reporting; alerting verified; Wazuh receiving.
- Offline secrets bundle refreshed to reflect any credential changes made during the rebuild.

Cross-check against Production-Readiness-Checklist in the vault, then record the rebuild in the change log (NF-GV-002).

12. Troubleshooting

Symptom	Action
Cannot unlock the secret store to get any other secret	You are in the circular dependency. Use the Phase 0 offline bundle; do not proceed with services until the secret store is restored and unlocked.
Cluster will not reach quorum on join	Verify corosync is on the management ring, time is synced, and no stale node entry remains from the old cluster. Add nodes one at a time.
No internet / no inter-VLAN after OPNsense build	Re-check the WAN1/LAN/WAN2 interface mapping; confirm the config import restored all seven VLAN interfaces and both outbound NATs.
Intermittent name resolution during rebuild	Expected while the edge resolver reloads (single recursion point). Confirm both Pi-holes forward to it; wait for reload to finish.
PBS restore fails to decrypt	Datastore is encrypted; supply the Phase 0 PBS encryption key and verify the fingerprint.
Kubernetes pods stuck pending on PVC	NFS provisioner or STORAGE NFS export is down; Phase 5 must be green before Phase 7 workloads schedule.
Registry image pulls fail on cold cluster	Registry runs on the cluster; pull base images externally until it serves valid TLS, then reconcile.
GPU not visible after driver install	Confirm the pinned kernel is the running kernel (not a newer one from dist-upgrade); the driver ABI is pinned to it. Hold the kernel.

See also: NF-RC-002 Disaster Recovery; NF-RC-001 Backup and Restore; NF-AR-001 System Architecture; NF-RF-010 IPAM; NF-RF-011 VLAN Reference; NF-RF-030 Naming Standards; NF-RF-031 Configuration Standards; NF-OP-001 Standard Operating Procedures. Detailed per-component runbooks live in the private Home-Lab operations vault.